12주차 강의 스크립트

자율 탐색 파이프라인 (1) — 파이프라인 구조와 데이터 큐레이션

건설소재 역설계 도구 개발을 위한 AI와 분자동역학 활용 · 윤태영 (Taeyoung Yun) · 강의 50분

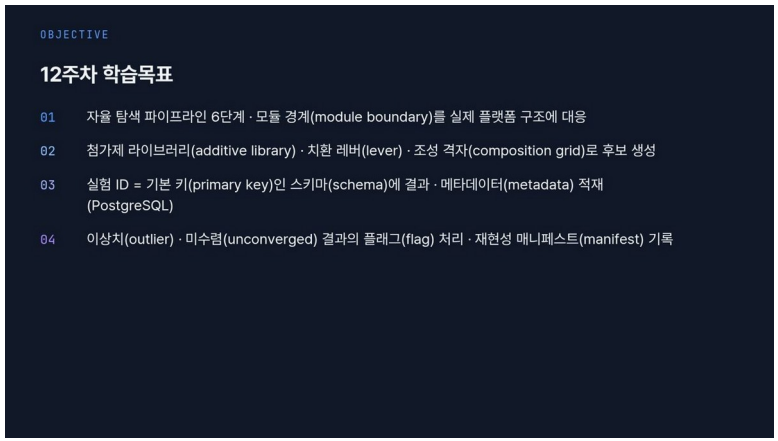
학습목표: 자율 탐색 파이프라인의 구성과 후보 생성·데이터베이스 큐레이션을 이해한다.

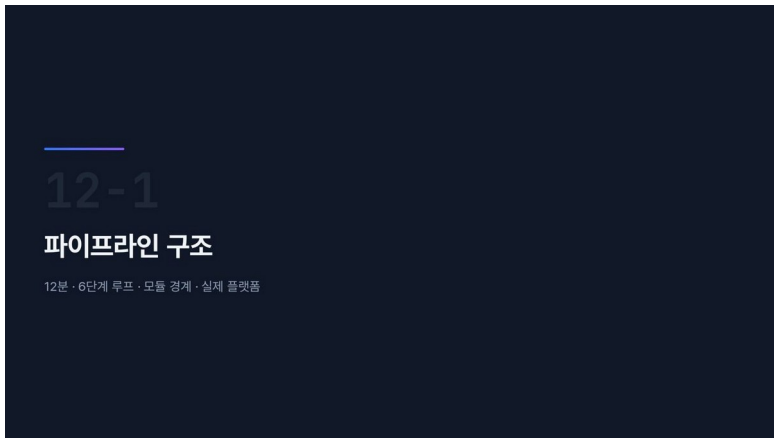
차시	차시명 (강의계획서)	시간	슬라이드
도입	도입 (표지 · 목표 · 구성)	2분	#1–#4
12-1	파이프라인 구조	11분	#5–#8
12-2	후보 분자 생성	11분	#9–#12
12-3	PostgreSQL 데이터 큐레이션	15분	#13–#16
12-4	이상치 처리·재현성	9분	#17–#20
마무리	실습 안내 · 정리	2분	#21–#22
합계		50분	25장

읽는 법

- 2 단 표: 왼쪽 = 화면(슬라이드 · 순번 · 차시 · 배정 시간 · 누적 시각) · 오른쪽 = 설명([진행] 지시 → 말할 내용)
- 【콘크리트 예시】 문단 = 시멘트 · 콘크리트 사례로 개념을 설명하는 부분 (녹색 배경)
- 부록 슬라이드는 50 분 발표 시간에 포함하지 않음 · 질문이 나올 때 참고용

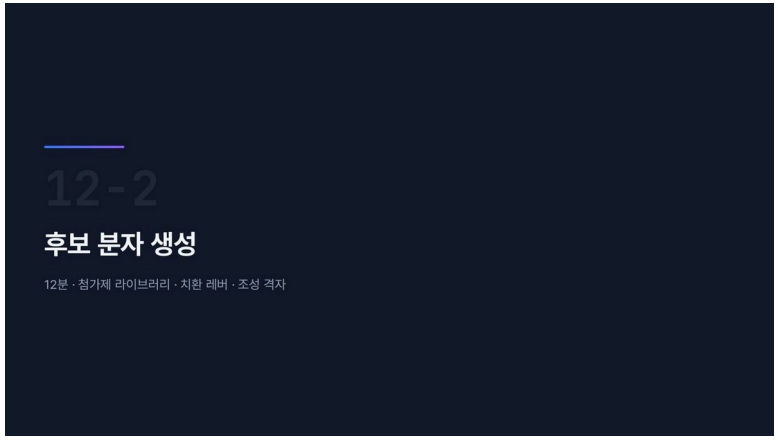
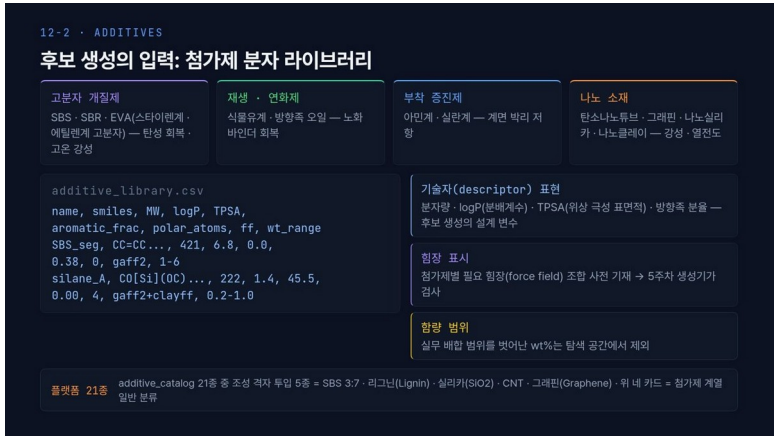
화면	설명
 **#01 표지** 도입 · 25초 0:00 → 0:25	12주차를 시작하겠습니다. 이번 주부터 자율 탐색 파이프라인 블록이고, 아래 줄처럼 파이프라인 구조, 후보 분자 생성, 데이터 큐레이션(data curation), 이상치 처리, 재현성을 다룹니다. 실습은 후보 격자와 데이터셋 구축입니다.
 **#02 지난 주차** 도입 · 35초 0:25 → 1:00	지금까지 쌓은 것을 되짚어 보겠습니다. 왼쪽 카드처럼 2–8주차의 조성별 · 온도별 물성값과 오차 · 궤적(trajectory), 9–10주차의 대리모델, 11주차의 문헌 후보가 있습니다. 문제는 가운데 카드입니다. 폴더에 흩어진 결과는 학습에 바로 쓸 수 없습니다. 그래서 이번 주에는 이것들을 한 루프로 묶는 구조를 세우고, 아래 줄처럼 후보 생성(01 → 02)과 데이터 적재(03 → 04)를 중심으로 봅니다.

화면	설명
 **#03 학습목표** 도입 · 40초 1:00 → 1:40	목표는 차시마다 하나씩입니다. 6단계와 모듈 경계(module boundary)를 실제 플랫폼에 대응하기, 라이브러리 · 치환 레버(lever) · 조성 격자(composition grid)로 후보 만들기, 실험 ID를 기본 키(primary key)로 하는 스키마(schema)에 적재하기, 이상치(outlier) · 미수렴 결과를 플래그로 처리하고 매니페스트(manifest) 남기기입니다.
 **#04 12주차 구성** 도입 · 20초 1:40 → 2:00	시간은 12-1 파이프라인 구조 12분, 12-2 후보 분자 생성 12분, 12-3 데이터 큐레이션 16분, 12-4 이상치 처리 · 재현성 10분 정도입니다. 가장 긴 것이 데이터베이스 차시입니다.

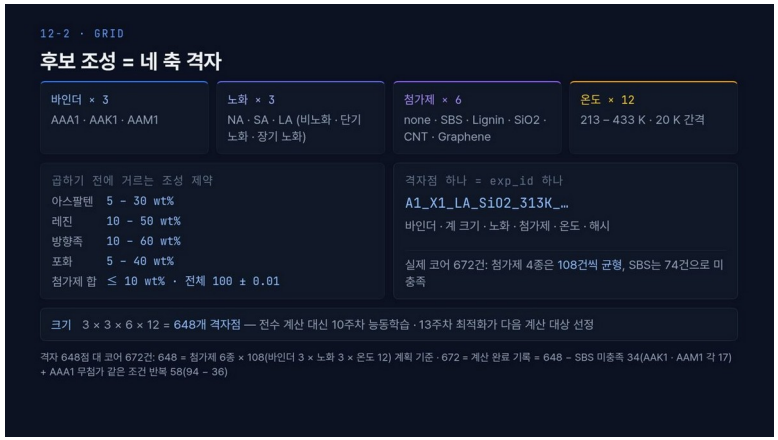
화면	설명
 **#05 [12-1] 섹션** 12-1 · 20초 2:00 → 2:20	첫 차시는 파이프라인 구조입니다. 1주차에 개요로 본 6단계 루프를, 구성 요소를 다 배운 지금 모듈 경계와 실제 코드 구조로 내려서 보겠습니다.
 **#06 파이프라인 6단계** 12-1 · 3분 30초 2:20 → 5:50	1주차에 처음 본 그림을 다시 가져왔습니다. 그때는 단계 이름만 들었지만, 지금은 칸마다 해당하는 주차를 모두 배웠습니다. 위 줄부터 보겠습니다. 01 문헌 고찰은 지난주에 한 일로, 후보 분자와 표적 범위를 모아 설계 변수를 정합니다. 02 구조 생성은 4주차의 분자 조립과 벌크 셀 구축으로, 조성을 계산 가능한 계로 만듭니다. 03 MD 계산은 5주차 실행부터 6–8주차 물성 · 소재 분석까지의 산출과 결과 적재이고, 여기서 데이터베이스 축적이 시작됩니다. 각 칸 오른쪽 위의 주차 표시를 보면, 03 MD 계산이 5–8주차로 가장 넓습니다. 바탕 이론인 2–3주차까지 치면 한 학기의 절반 이상을 계산 방법에 썼다는 뜻이고, 그만큼 이 단계에서 나오는 결과의 종류와 형식이 다양합니다. 밀도처럼 숫자 하나로 끝나는 것도 있고, 응력-변형률처럼 곡선으로 나오는 것도 있고, 수렴이 안 돼 실패한 계산도 있습니다. 이 다양한 결과를 04 대리모델이 읽을 수 있는 한 가지 형식으로 모으는 것이 이번 주 12-3의 일입니다. 아래 줄은 학습 쪽입니다. 04 대리모델은 9–10주차의 XGBoost+SHAP(Shapley 값으로 특징 기여도를 나누는 방법)과 GNN, 그리고 불확실성으로, MD 한 번보다 훨씬 빨리 물성을 추정하는 평가 함수를 줍니다. 05 능동학습은 10주차에 본 대로 다음 계산 대상을 골라 계산 예산을 배분합니다. 06 최적화는 13주차에 다룰 다목적 최적화와 특징 재정련으로, 추천 조성과

화면	설명
	개선된 표현을 내놓습니다. 아래 회색 줄이 중요합니다. 06의 출력이 다시 01의 입력으로 돌아갑니다. 최적화가 추천한 조성이 다시 구조 생성과 MD로 들어가고, 그 결과가 쌓여 대리모델을 다시 학습시킵니다. 파이프라인은 직선이 아니라 루프입니다. 여기서 많이 헷갈리는 것이, 루프니까 단계들이 알아서 맞물린다고 생각하는 점입니다. 실제로는 단계 사이에서 넘어가는 데이터의 형식이 맞지 않으면 루프가 거기서 끊깁니다. 그래서 맨 아래 파란 줄처럼 이번 주는 두 이음매를 다룹니다. 01 → 02 사이, 문헌 후보를 계산할 조성 목록으로 바꾸는 후보 생성과, 03 → 04 사이, MD 결과를 학습 가능한 표로 바꾸는 데이터 적재입니다.
 **#07 통합 구조** 12-1 · 3분 15초 5:50 → 9:05	앞 장의 6단계를 소프트웨어 모듈 여섯 개로 바꿔 그리면 이렇습니다. 왼쪽부터 builder는 4주차의 분자 · 셀 구성, runner는 5주차의 계산 실행, analyzer는 6–8주차의 물성 분석, store는 이번 12주차의 저장, model은 9–10주차의 대리모델, optimizer는 10 · 13주차의 능동학습 · 최적화입니다. 빨간 점선처럼 optimizer가 다음 조성을 내면 다시 builder로 돌아갑니다. 앞 장의 루프가 코드에서는 이 화살표입니다. 아래 세 카드가 설계 원칙입니다. 첫째 인터페이스 고정. 모듈끼리는 파일이나 DB 레코드로만 주고받고, 내부 구현은 자유입니다. 예를 들어 analyzer가 점도를 Green-Kubo로 구하든 비평형 MD로 구하든, store에 넘기는 레코드 형식만 같으면 뒤쪽 model은 고칠 필요가 없습니다. 반대로 모듈끼리 서로의 함수를 직접 부르기 시작하면 한 곳을 고칠 때 여러 곳이 같이 깨집니다. 예를 들어 YAML 파일에는 「표적: 점도와 밀도, 제약: 첨가제 합 10 wt% 이하, 예산: GPU 시간, 티어: 선별용」 같은 항목이 들어갑니다. 같은 파일로 다시 돌리면 같은 계획이 실행되므로, 결과가 달라졌을 때 설정이 바뀐 것인지 코드가 바뀐 것인지를 먼저 가를 수 있습니다. 둘째 설정 주도. YAML 설정 파일 하나에 표적 물성 · 제약 · 계산 예산 · 티어(tier)를 적으면 전체가 실행됩니다. 티어는 계산 정밀도 등급으로, 빠른 선별용 짧은 계산과 정밀한 긴 계산을

화면	설명
	구분하는 표시입니다. 설정을 파일로 남기면 그 자체가 실행 기록이 됩니다. 셋째 부분 실행. 단계마다 단독으로 돌릴 수 있어야 합니다. 결과 파싱이 틀렸을 때 셀을 다시 만들고 MD를 다시 돌릴 필요 없이 analyzer만 다시 돌리면 됩니다. 이 여섯 칸 가운데 이번 주의 몫은 보라색 store입니다. 다음 장에서 이 모듈들이 실제 플랫폼 코드에서 어떻게 나뉘어 있는지 보겠습니다.
 **#08 실제 플랫폼 구조** 12-1 · 3분 55초 9:05 → 13:00	실제 COMID 플랫폼 구성입니다. 위쪽 다섯 상자를 요청 · 조율 · 실행 · 저장 순서로 읽으면 됩니다. 맨 왼쪽 Frontend는 React SPA로 만든 웹 화면으로 분자 · 계면 빌더, GPU 모니터, 차트가 있고, 여기서 한 요청이 HTTP로 Backend에 갑니다. Backend는 FastAPI REST 서버와 Orchestrator로, Orchestrator가 GPU 자원과 작업 간 의존성을 보고 순서를 정한 뒤 Celery와 Redis 큐로 작업을 넘깁니다. 큐(queue)는 작업을 줄 세워 두었다가 자원이 비면 꺼내 주는 대기열입니다. 가운데 Execution이 실제 계산입니다. Packmol로 분자를 상자에 채우고, antechamber · parmchk2로 GAFF 계열 힘장 매개변수를 붙이고, LAMMPS를 KOKKOS+CUDA로 GPU에서 돌립니다. MPS는 GPU 하나에 여러 작업을 동시에 올리는 기능입니다. Core 상자의 builder · typing_router · protocols(Jinja2 템플릿) · parsers · metrics는 회색 글씨처럼 GPU 없이도 검토 · 시험할 수 있게 따로 묶여 있습니다. 앞 장의 인터페이스 고정이 이렇게 구현된 것입니다. 맨 오른쪽 Store는 SQLAlchemy로 접근하는 DB와 결과 파일입니다. 맨 위 점선 상자 contracts · policies · common hashing은 모든 모듈이 따르는 규칙층입니다. Backend와 Core가 같은 규칙을 쓰므로 모듈이 바뀌어도 실험 ID를 만드는 해시와 정책은 하나로 유지됩니다. 같은 조성을 두 곳에서 다른 이름으로 부르는 일을 막는 장치입니다. 아래 칩 줄은 실험 한 건의 흐름입니다. 조성 명세 → 셀 구축 → 힘장 할당 → 프로토콜 · 티어 → GPU 슬롯 → LAMMPS → 로그 파싱 → 평균 ± SE(표준오차) → 구조화 DB. 마지막 보라 칩이 이번 주의 도착점입니다. 노란 줄처럼 ML 단계는 이 구조화 DB를 읽는 대리모델, 9주차 XGBoost+SHAP과 10주차 GNN이고, 능동학습이 이 모델로 다음 계산을 고릅니다. 여기서 많이 틀리는 것이 ML 코드가 LAMMPS 로그를 직접 읽게 만드는 것입니다. 그러면 파싱 규칙이

화면	설명
	바뀔 때마다 학습 코드도 고쳐야 합니다. 반드시 구조화 DB를 거칩니다. 구조를 봤으니 12-2에서는 이 루프에 넣을 후보를 만드는 규칙으로 넘어가겠습니다.
 **#09 [12-2] 섹션** 12-2 · 20초 13:00 → 13:20	두 번째 차시는 후보 분자 생성입니다. 계산할 후보를 어떤 규칙으로 만드는지, 첨가제 라이브러리 · 치환 레버 · 조성 격자 세 가지로 보겠습니다.
 **#10 첨가제 라이브러리**	후보 생성의 입력은 7주차에 만든 첨가제 분자 라이브러리입니다. 위 네 카드가 분류입니다. 고분자 개질제는 SBS · SBR · EVA 같은 스타이렌계 · 에틸렌계 고분자로 탄성 회복과 고온 강성을 올립니다. 재생 · 연화제는 식물유계 · 방향족 오일로 노화된 바인더를 회복시킵니다. 부착 증진제는 아민계 · 실란계로 계면 박리 저항을 높이고, 나노 소재는 탄소나노튜브 · 그래핀 · 나노실리카 · 나노클레이로 강성과 열전도를 바꿉니다. 네 카드는 계열 분류이고, 맨 아래 줄처럼 플랫폼 카탈로그 21종 가운데 조성 격자에 실제로 들어가는 것은 SBS 3:7, 리그닌, 실리카, CNT, 그래핀 5종입니다. 왼쪽 아래 additive_library.csv가 실제 형식입니다. 열은 name, smiles, MW(분자량), logP, TPSA, aromatic_frac, polar_atoms, ff, wt_range입니다. 첫 행 SBS_seg는 분자량 421, logP 6.8, TPSA 0.0, 방향족 분율 0.38, 극성 원자 0, 힘장 gaff2, 함량 1–6 wt%이고, 둘째 행 silane_A는 분자량 222, logP 1.4, TPSA 45.5, 극성 원자 4, 힘장 gaff2+clayff, 함량 0.2–1.0

화면	설명
12-2 · 3분 20초 13:20 → 16:40	wt%입니다. 두 행을 비교하면 SBS 세그먼트는 극성이 없는 소수성 분자이고, 실란은 극성 표면적이 크고 광물 표면과 붙는 쪽이라 무기물 힘장 ClayFF가 함께 필요하다는 것이 숫자로 드러납니다. 오른쪽 세 카드입니다. 기술자(descriptor) 표현은 분자를 분자량 · logP(분배계수, 기름과 물 중 어디에 잘 녹는지) · TPSA(위상 극성 표면적) · 방향족 분율 같은 숫자로 바꿔 설계 변수로 쓰는 것입니다. 힘장 표시는 첨가제마다 필요한 힘장 조합을 미리 적어 5주차 생성기가 검사하게 하는 것입니다. 이 칸이 비어 있으면 셀을 다 만든 뒤 힘장 할당 단계에서야 실패가 드러나 GPU 시간이 낭비됩니다. 함량 범위는 실무 배합 범위를 벗어난 wt%를 탐색 공간에서 빼는 규칙입니다. 계산으로는 얼마든지 많이 넣을 수 있지만, 실제로 만들 수 없는 배합이 최적해로 나오면 쓸모가 없습니다. 다음 두 장에서 이 라이브러리에 치환 규칙과 조성 격자를 적용합니다.
 **#11 구조 치환 레버** 12-2 · 3분 45초 16:40 → 20:25	k 레버의 알킬사슬은 아스팔텐 가장자리에 붙는 긴 탄화수소 사슬이고, s 레버의 티오에터는 탄소 사이에 황이 끼어 있는 C–S–C 결합입니다. 황 함량은 바인더 원산지마다 크게 달라서, 측정된 황 함량을 맞추려면 황이 들어갈 자리를 레버로 조절할 수 있어야 합니다. 그래도 모자라 13번째 분자를 더한 것입니다. 후보를 만드는 첫 번째 규칙은 치환입니다. 새 분자를 처음부터 그리지 않고, 검증된 골격을 고정한 채 정해진 자리만 바꿉니다. 왼쪽 01 라이브러리는 Li & Greenfield 12분자 골격으로, 포화 2 · 방향족 2 · 레진 5 · 아스팔텐 3개이고 고리계와 헤테로원자는 유지합니다. 아래 13번째 고황 아스팔텐 C47H68S3은 12분자만으로 황 함량을 맞추기 어려워 더한 분자입니다. 가운데 02 정수 레버 격자입니다. k는 아스팔텐 알킬사슬 {0, 12, 16}, a는 방향족 메틸치환 {0, 2}, s는 티오에터(thioether) 추가 {0, 1, 2}입니다. 3 × 2 × 3 = 18개 격자점이고, 격자점마다 분자 개수를 최적화합니다. 레버를 정수로 둔 이유는 화학적으로 의미 있는 변형만 허용하려는 것입니다. 메틸기 1.5개 같은 값은 분자로 존재하지 않습니다. 오른쪽 03은 생성 직후 검사입니다. 화학 안정성은 라디칼 0개, 긴장고리(strained ring) 없음, 합성 가능성을 봅니다. SARA(포화분 · 방향족 · 레진 · 아스팔텐) 역할 검사는 바뀐 분자가 여전히 자기 분류에 맞는지 봅니다. 아스팔텐이라면 다환방향족 + 헤테로원자 + 알킬곁사슬을

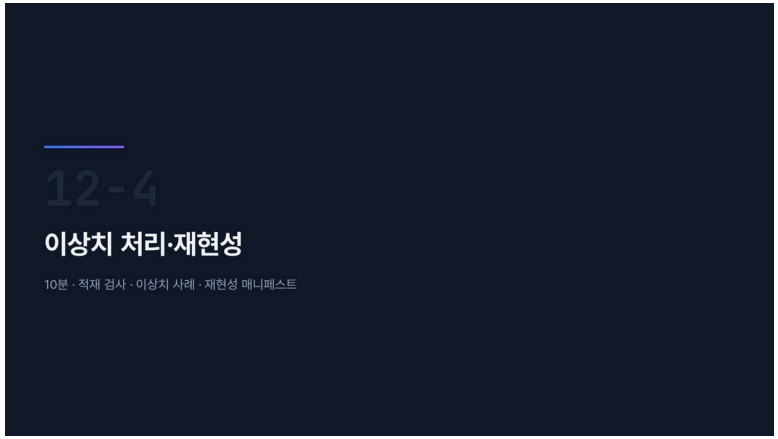
화면	설명
	갖춰야 합니다. 떨어진 조합은 후보에서 뺍니다. 여기서 많이 틀리는 것이 레버를 적용하면 분자량만 바뀐다고 생각하는 것입니다. 치환 때문에 분자가 원래 SARA 분류의 구조 조건을 벗어날 수 있고, 역할 검사를 빼면 조성 계산 자체가 틀어집니다. 아래 절차 칩이 전체 순서입니다. 골격 분자 → RDKit로 레버 적용 → 안정성 · SARA 검사 → 분자 개수 최적화 → 후보 조성. 맨 아래 결과 예를 보면 PG(Performance Grade, 공용성 등급) 64-22 · 76-22 · 82-22의 최적 레버 (k, a, s)가 각각 (16,2,1), (16,2,0), (16,0,2)입니다. 세 등급 모두 k는 16으로 같지만 a와 s가 달라, 등급마다 다른 후보가 나온다는 것을 보여 줍니다.
 **#12 조성 격자** 12-2 · 3분 35초 20:25 → 24:00	온도 축 213–433 K는 섭씨로 대략 −60 °C부터 160 °C까지로, 저온 취성 영역부터 가열 혼합 온도 부근까지 걸칩니다. 20 K 간격 12점이면 유리전이 부근의 곡선 꺾임도 몇 개 점으로 잡을 수 있습니다. 두 번째 규칙은 조성 격자(composition grid)입니다. 위 네 카드가 축입니다. 바인더 3종 AAA1 · AAK1 · AAM1, 노화 3단계 NA · SA · LA(비노화 · 단기 · 장기 노화), 첨가제 6종 none · SBS · Lignin · SiO2 · CNT · Graphene, 온도 12점 213–433 K · 20 K 간격입니다. 아래 줄처럼 3 × 3 × 6 × 12 = 648개 격자점입니다. 왼쪽 가운데 상자는 곱하기 전에 거르는 조성 제약입니다. 아스팔텐 5–30 wt%, 레진 10–50, 방향족 10–60, 포화 5–40 wt%, 첨가제 합 10 wt% 이하, 전체 100 ± 0.01 wt%입니다. 순서가 중요합니다. 격자를 다 만든 뒤 거르는 것이 아니라 생성 단계에서 제약 밖 조합을 아예 만들지 않습니다. 만든 뒤 거르면 제약 밖 조합이 실수로 계산 대기열에 들어가는 일이 생깁니다. 오른쪽 상자는 격자점 하나가 exp_id 하나라는 뜻입니다. A1_X1_LA_SiO2_313K_…처럼 바인더 · 계 크기 · 노화 · 첨가제 · 온도 · 해시가 이름에 들어가, 이름만 보고도 어떤 계산인지 알고 끝의 해시로 같은 조건의 중복 계산을 막습니다. 그 아래 실제 현황은 플랫폼 코어 672건에서 첨가제 4종은 108건씩 균형이고 SBS는 74건으로 덜 찼다는 것입니다. 격자를 기준으로 두면 빈 곳이 이렇게 바로 보입니다. 맨 아래 줄처럼 648은 계획이고 672는 실제 기록이라, SBS 빈칸 34건을 빼고 AAA1 무첨가 반복 계산 58건을 더하면 672가 됩니다. 648

화면	설명
	점을 모두 계산하지는 않습니다. 10주차 능동학습과 13주차 최적화가 다음 대상을 고르고, 격자는 선택 가능한 후보의 범위입니다. **【콘크리트 예시】** 콘크리트 배합 설계도 같은 구조입니다. 물시멘트비(예: 대략 0.4–0.5 부근의 몇 단계), 혼화재 종류(없음 · 플라이애시 · 슬래그), 치환율, 재령(7 · 28일)을 축으로 두고, 단위 결합재량 같은 제약을 먼저 적용해 격자를 만듭니다. 격자점마다 배합 ID를 붙이면 다음 차시에 볼 데이터베이스의 키가 됩니다.
 **#13 [12-3] 섹션** 12-3 · 25초 24:00 → 24:25	세 번째 차시는 데이터 큐레이션입니다. 세 테이블 설계, 실제 플랫폼 ERD, 적재 계약 순서로 봅니다. 미리 말씀드리면 플랫폼의 운영 DB는 SQLite이고, PostgreSQL은 같은 설계를 옮기는 방법으로 다룹니다.

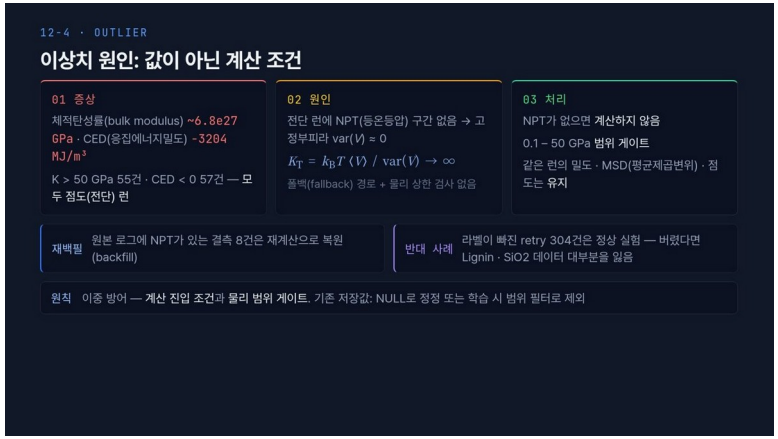
화면	설명
 **#14 스키마 설계** 12-3 · 4분 35초 24:25 → 29:00	조성 · 조건 · 물성을 엑셀 한 장에 다 넣으면 처음에는 편하지만 곧 문제가 생깁니다. 같은 조성을 온도 12점에서 계산하면 조성 정보가 12번 반복되고, 그중 한 줄만 고치면 나머지와 어긋납니다. 그래서 세 테이블로 나눕니다. 위 그림 왼쪽 composition은 comp_id와 분자별 개수, 첨가제와 함량 wt%를 담습니다. 가운데 run은 실행 한 건으로 exp_id, 어떤 조성인지 가리키는 comp_id, 힘장, 온도 T, 시드, 해시, 상태를 담습니다. 오른쪽 property는 exp_id, 물성명, 값, 단위, 오차, 수렴 플래그입니다. 가운데 회색 글씨처럼 한 조성 → 여러 실행 → 실행마다 여러 물성, 1:N:N 관계입니다. 실제로 쓰는 모습을 떠올려 보겠습니다. 「AAA1 바인더 · SBS 첨가 조성의 413 K 점도」를 찾으려면 composition에서 조성을 고르고, run에서 그 comp_id와 온도로 실행을 찾고, property에서 그 exp_id의 점도 행을 가져옵니다. 세 테이블을 키로 잇는 조인(join)입니다. 조성 정보는 한 곳에만 있으니, 첨가제 함량 오타를 고칠 때도 한 줄만 고치면 모든 실행에 반영됩니다. 아래 세 카드가 이유입니다. 왜 나누나: 조성은 재사용되고 실행은 반복되므로 조성을 한 번만 적어 중복을 줄이고 정합성을 지킵니다. 키 규칙: exp_id는 4주차의 실험 ID 문자열을 그대로 쓰고 해시를 포함합니다. DB 안에서 새 번호를 따로 매기면 결과 폴더와 DB 행을 잇는 고리가 끊깁니다. 긴 형식(long format): 물성을 열이 아니라 행으로 둡니다. 밀도 · 점도 · CED를 각각 열로 만들면 새 물성이 생길 때마다 테이블 구조를 바꿔야 하지만, 「exp_id, 물성명, 값」 행으로 두면 행만 추가하면 됩니다. 여기서 많이 헷갈리는 것이, 학습할 때는 결국 넓은 표가 필요하지 않느냐는 점입니다. 맞습니다. 저장은 긴 형식으로 하고, 학습 직전에 피벗(pivot)으로 넓은 표로 펼칩니다. 저장 형식과 학습 형식을 구분하는 것이 핵심입니다. **【콘크리트 예시】** 콘크리트 시험 데이터도 똑같이 나눕니다. mix 테이블에 배합 ID · 물시멘트비 · 단위 시멘트량 · 혼화재 치환율을, specimen 테이블에 공시체 ID · 배합 ID · 타설일 · 양생 조건 · 형상을, test 테이블에 공시체 ID · 시험 종류 · 재령 · 값 · 단위를 둡니다. 한 배합에 공시체 여러 개, 공시체마다 시험이 있으니 역시 1:N:N입니다. 7일 · 28일 강도를 열로 두지 않고 재령을 행으로 두면, 나중에 91일 시험이 추가돼도 구조를 바꿀 필요가

화면	설명
	없습니다.
 **#15 플랫폼 ERD** 12-3 · 4분 55초 29:00 → 33:55	앞 장의 기본형이 실제 플랫폼에서는 이렇게 넓어집니다. 개체-관계도(ERD, entity-relationship diagram)는 테이블을 상자로, 테이블 사이의 관계를 선으로 그린 그림이고, 오른쪽 아래 글씨처럼 이 그림의 모든 선은 1 : N입니다. 가운데 파란 EXPERIMENT가 실행 한 건입니다. 기본 키 exp_id (PK), composition_hash, plan_hash, temperature_k, status가 있습니다. composition_hash는 조성의 지문, plan_hash는 계산 절차의 지문입니다. 둘을 나눠 두면 「같은 조성 · 다른 절차」와 「다른 조성 · 같은 절차」를 쉽게 골라낼 수 있습니다. 왼쪽 초록 상자 두 개가 조성입니다. 앞 장의 composition 하나가 여기서는 MOLECULE과 EXPERIMENT_MOLECULE 연결 테이블로 분자 단위까지 내려갑니다. 한 실험에 여러 분자가, 한 분자가 여러 실험에 쓰이는 다대다 관계를 연결 테이블로 푼 것입니다. 그래서 「이 아스팔텐 분자가 들어간 실험 전부」 같은 질의가 간단해집니다. 아래 세 상자는 결과입니다. METRIC은 name · unit · value · uncertainty로 스칼라 물성과 불확실도를 담습니다. ARRAY_CURVE는 name과 parquet_path만 있습니다. 응력-변형률 곡선처럼 점이 수천 개인 배열은 DB에 넣지 않고 parquet 파일로 두고 경로만 적습니다. PROVENANCE, 즉 계보는 결과가 어떤 입력 · 코드 · 절차에서 나왔는지를 따로 쌓습니다. PROVENANCE 테이블이 왜 따로 있어야 하는지 12-4의 이상치 사례와 연결해 보겠습니다. 어떤 값이 이상할 때 「이 값은 어떤 파서 버전으로, 어떤 로그 구간에서 계산했는가」를 찾을 수 있어야 같은 원인으로 오염된 다른 값들을 한꺼번에 골라낼 수 있습니다. 계보가 없으면 이상한 값을 하나씩 눈으로 찾는 수밖에 없습니다. 오른쪽 카드가 운영 규칙입니다. 1급 컬럼: ML 특징, 쿼리 필터, 계보 판정에 쓰는 값은 모두 정식 컬럼으로 올립니다. metadata_json: 사람 메모 · 디버깅 정보 · 임시 태그만 두고, 정형화할 수 있으면 컬럼으로 승격합니다. 여기서 많이 틀리는 것이 편하다고 온도나 첨가제 종류를 JSON 안에 넣는 것입니다. 그러면 「433 K 실험만」 같은 필터가 느려지고, 누군가 키 이름을 temp로 바꿔 적는 순간 그 행은 필터에서 사라집니다. 11주차 추출 결과에서 본 표기

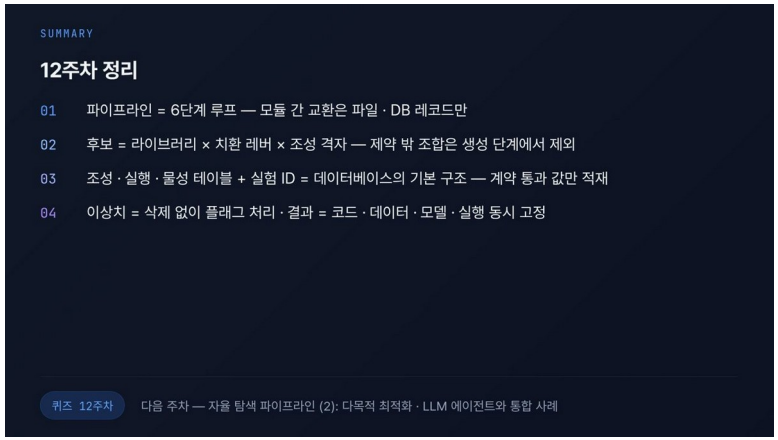
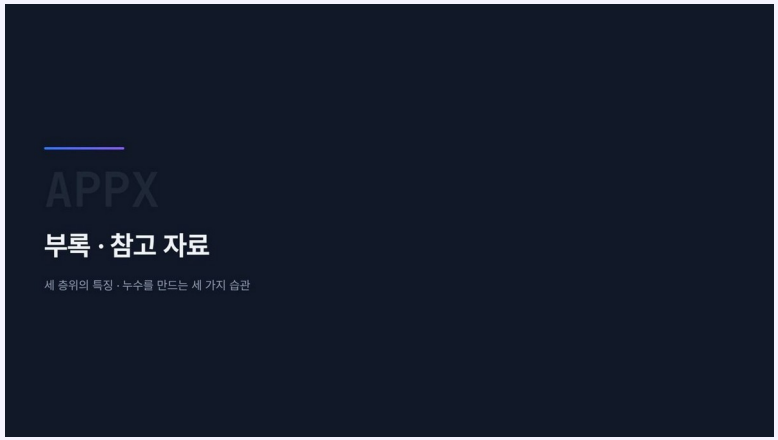
화면	설명
	분산이 DB 안에서도 똑같이 생기는 것입니다. 곡선은 파일로: DB는 작고 빠르게 두고 큰 배열은 파일로 뺍니다. 참고로 이 플랫폼 운영 DB는 SQLite이고 PostgreSQL로 운영한 산출물은 아직 없습니다. 다음 장에서 이 설계를 PostgreSQL로 옮기면 무엇이 좋아지는지 보겠습니다.
 **#16 PostgreSQL 적재 계약** 12-3 · 5분 5초 33:55 → 39:00	데이터베이스가 있어도 아무 값이나 들어가면 소용이 없습니다. 그래서 적재는 계약(contract)을 통과한 값만 받습니다. 왼쪽 첫 카드 fail-closed 저장은, 규칙을 어기면 열어 주지 않고 닫히는 쪽으로 동작한다는 뜻입니다. 물성 이름은 레지스트리(registry, 등록부)에 등록된 것만 받고, 단위와 namespace가 정의와 다르면 저장을 거부합니다. 점도를 「visc」로 적거나 등록과 다른 단위로 넣으면 경고만 하고 넘어가는 것이 아니라 오류로 멈춥니다. 이전 자료를 옮길 때만 예외를 명시적으로 켭니다. 경고만 띄우는 방식은 결국 아무도 경고를 읽지 않게 되기 때문입니다. 둘째 카드 SSOT(single source of truth, 단일 진실원본)는 학습 · 서빙 · 역설계가 모두 피처 생성기 하나만 쓴다는 규칙입니다. 학습과 예측에서 특징을 서로 다른 코드로 만들면 같은 분자가 조금씩 다른 숫자가 되어 모델이 조용히 틀립니다. 셋째 카드 스키마 해시는 피처 구성이 바뀌면 해시가 바뀌고, 모델에 저장된 해시와 다르면 서빙을 거부하는 장치입니다. 특징 열 순서 하나만 바뀌어도 잡아냅니다. 오른쪽은 PostgreSQL로 옮긴 강의용 DDL 예시입니다. experiment 테이블에 exp_id text PRIMARY KEY, comp_asphaltene_wt · temperature_k real NOT NULL, run_tier · status NOT NULL, metadata_json jsonb가 있습니다. jsonb는 PostgreSQL의 이진 JSON 형식으로 안쪽 키를 검색 · 색인할 수 있습니다. metric 테이블의 exp_id는 REFERENCES experiment, name은 REFERENCES metric_registry입니다. 이 두 줄이 외래 키(foreign key)로, 없는 실험의 물성이나 등록 안 된 물성 이름을 DB가 스스로 거부합니다. NOT NULL은 온도가 빠진 실험을 받지 않게 합니다. 맨 아래 줄이 핵심 차이입니다. 운영 DB인 SQLite와 SQLAlchemy에서는 레지스트리 검사를 파이썬 코드가 합니다. PostgreSQL로 옮기면 그 검사를 외래 키 · CHECK 제약으로 DB가 직접 수행하므로, 누가 어떤 스크립트로 넣든 같은 규칙이 적용됩니다. 여기서 많이 오해하는

화면	설명
	것이 PostgreSQL로 바꾸기만 하면 품질이 좋아진다는 생각입니다. 제약을 선언해야만 DB가 지켜 줍니다. **【콘크리트 예시】** 콘크리트 test 테이블이라면 시험 종류는 등록된 시험 목록을 외래 키로 참조하고, 강도값에 CHECK (value > 0), 물시멘트비에 CHECK (w_c > 0 AND w_c < 1) 같은 명백한 물리 범위를 겁니다. 재령과 단위도 NOT NULL로 두면 재령이 빠진 강도값이 섞여 28일 강도 평균을 흐리는 일을 막습니다. 이것으로 12-3을 마치고, 12-4에서는 계약을 통과한 값 중에서도 이상한 값을 어떻게 다루는지 보겠습니다.
 **#17 [12-4] 섹션** 12-4 · 20초 39:00 → 39:20	마지막 차시는 이상치 처리와 재현성입니다. 이상한 값과 실패한 실행을 어떻게 표시하고, 결과를 어떻게 다시 만들 수 있게 남기는지 봅니다.

화면	설명
 **#18 품질 점검** 12-4 · 2분 40초 39:20 → 42:00	기준값의 의미도 짚어 두겠습니다. 아스팔트 바인더의 밀도는 대략 1 g/cm³ 부근이라 0.7–1.3 범위를 벗어나면 셀이 제대로 압축되지 않았거나 단위 변환이 틀렸을 가능성이 큽니다. 적분 평탄 구간이란, 점도를 Green-Kubo로 구할 때 상관함수 적분값이 시간에 따라 일정한 값으로 수렴하는 구간을 말합니다. 평탄 구간 없이 계속 오르내리면 계산 시간이 모자랐다는 뜻이라 그 값은 믿을 수 없습니다. 이 기준들은 사람이 눈으로 보던 것을 코드로 옮긴 것입니다. 적재 시점에 자동으로 도는 검사 다섯 가지입니다. 첫 줄 물리 범위는 밀도가 0.7–1.3 g/cm³ 안인지, 점도가 음수가 아닌지 보고, 어기면 invalid 플래그입니다. 둘째 수렴은 평형 판정 통과와, Green-Kubo 적분처럼 적분 곡선에 평탄 구간이 있는지 보고, 없으면 unconverged 플래그입니다. 셋째 재현성은 시드 간 편차가 보고 오차의 3배 이내인지 보고, 넘으면 재계산 대기열로 보냅니다. 넷째 중복은 같은 해시가 두 번 적재됐는지 보고 최신 것만 유지합니다. 다섯째 단위는 단위 문자열이 사전과 맞는지 보고, 틀리면 적재를 거부합니다. 처리 방식이 다르다는 점을 보십시오. 단위 오류는 거부, 범위 · 수렴 문제는 플래그, 재현성 문제는 재계산입니다. 단위가 틀린 값은 바로잡을 근거가 없지만, 범위를 벗어난 값은 왜 벗어났는지 자체가 정보이기 때문입니다. 아래 원칙 줄이 이 차시의 핵심입니다. 실패 결과도 지우지 않고 플래그만 붙입니다. 10주차 보충 B의 능동학습은 이 플래그로 「어떤 조건에서 계산이 깨지는가」까지 학습합니다. 실패를 지우면 그 영역에 대한 정보가 사라집니다. 다음 장은 이 검사가 없어서 생긴 실제 사례입니다.

화면	설명
 **#19 이상치 사례** 12-4 · 3분 55초 42:00 → 45:55	감각을 잡기 위해 비교해 보면, 물의 체적탄성률이 대략 2 GPa입니다. 오른쪽 처리 카드의 0.1–50 GPa 게이트는 이 정도 크기를 넉넉히 감싸는 물리 범위이고, 6.8e27 GPa는 어떤 물질에서도 나올 수 없는 값입니다. 이런 값이 학습 데이터에 한 건만 섞여도 평균과 분산이 망가져 모델 전체가 영향을 받습니다. 왼쪽 01 증상입니다. 체적탄성률(bulk modulus)이 약 6.8e27 GPa, CED가 −3204 MJ/m³로 저장돼 있었습니다. 10의 27제곱 GPa의 체적탄성률이나 음수 응집에너지밀도는 물리적으로 불가능합니다. 화면처럼 체적탄성률 50 GPa 초과 55건과 CED 음수 57건이 모두 점도를 구하는 전단 런에서 나왔습니다. 가운데 02 원인입니다. 전단 런에는 NPT(등온등압) 구간이 없고 부피가 고정돼 있어 부피 분산 var(V)가 거의 0입니다. 부피 요동으로 체적탄성률을 구하는 식 K_T = k_B T ⟨V⟩ / var(V)는 분모가 0으로 가면 발산합니다. 여기에 NPT가 없으면 전체 로그로 넘어가는 폴백(fallback) 경로가 있었고 물리 상한 검사도 없어 그대로 저장됐습니다. 제목처럼 원인은 값이 아니라 계산 조건입니다. 많이 틀리는 것이, 이런 값을 평균 ± 3σ 밖 제거 같은 통계 처리로 해결하려는 것입니다. 원인이 남아 다음 계산에서 또 나옵니다. 오른쪽 03 처리는 이중 방어입니다. NPT가 없으면 아예 계산하지 않고, 0.1–50 GPa 범위 게이트를 둡니다. 같은 런의 밀도 · MSD(평균제곱변위) · 점도는 유효하므로 런 전체를 버리지 않습니다. 이미 저장된 값은 원칙 줄처럼 NULL로 정정하거나 학습 때 범위 필터로 뺍니다. 아래 두 카드는 반대 방향의 교훈입니다. 재백필(backfill)은 원본 로그에 NPT가 있는데 결측이던 8건을 재계산으로 채운 것이고, 라벨이 빠진 retry 304건은 버릴 뻔했지만 실제로는 정상 완료된 실험이었고, 재제출 함수가 material_id를 고정값으로 넣는 버그 때문에 라벨만 잃은 것이었습니다. 결과 경로 문자열로 바인더를 복원했습니다. 버렸다면 Lignin · SiO2 데이터 대부분을 잃었을 것입니다. **【콘크리트 예시】** 공시체 압축강도도 같습니다. 같은 배합의 공시체 여러 개(흔히 3개) 중 하나만 유독 낮다면 먼저 캡핑 · 연마 상태, 공시체 결함, 파괴 형태 같은 시험 조건을 확인합니다. 원인이 확인되면 그 값에 플래그를 붙여 평균에서 빼고, 원인이 없으면 값을 남긴 채 기록합니다. 값만 보고 지우지 않습니다.

화면	설명
 **#20 재현성** 12-4 · 2분 5초 45:55 → 48:00	앞의 사례처럼 값을 나중에 다시 계산하려면 그 값이 어떤 코드 · 데이터 · 모델 · 실행 조건에서 나왔는지 남아 있어야 합니다. 위 네 카드가 함께 고정할 것입니다. 코드는 git 커밋 해시와 conda · requirements 환경 파일, 데이터는 스냅샷 해시와 폴드 인덱스 파일, 모델은 체크포인트 · 하이퍼파라미터 · 난수 시드, 실행은 LAMMPS 버전 · 힘장 파일 · 입력 해시입니다. 매니페스트 각 줄의 쓰임을 보겠습니다. code의 git 해시로 그때의 코드를 그대로 꺼낼 수 있고, env 파일로 라이브러리 버전을 맞춥니다. data의 sha256은 데이터 파일 내용으로 만든 지문이라, 한 행만 바뀌어도 값이 달라져 「같은 데이터인가」를 바로 판정합니다. model의 seed는 난수 초기값이고, run의 LAMMPS 버전과 힘장 파일은 같은 입력이라도 버전이 다르면 결과가 조금씩 달라질 수 있어서 적습니다. 가운데 run_manifest.yaml이 기록 형식이고, 값은 형식을 보여 주는 예시입니다. 많이 빠뜨리는 것이 데이터 스냅샷입니다. 코드와 시드를 고정해도 DB가 그사이 바뀌면 같은 결과가 안 나옵니다. 방금 본 백필과 NULL 정정이 바로 DB를 바꾸는 작업입니다. 아래 줄처럼 이번 실습부터 모든 결과에 매니페스트를 붙이고, 13주차 최종 제출물에도 필수입니다.
 **#21 실습 안내** 마무리 · 1분	**[진행]** *퀴즈 12주차 배지를 가리키며 퀴즈 안내.* 실습은 후보 격자와 학습용 데이터셋 구축입니다. 01 격자를 만들고 조성 제약으로 거르고, 02 6–8주차 결과를 세 테이블로 적재합니다. SQLite · PostgreSQL · parquet 중 편한 것을 쓰셔도 됩니다. 03 RDKit 기술자로 조성 가중 평균을 만들고, 04 comp_id 기준 5-폴드로 나눠 폴드별 μ · σ를 저장합니다. 03 · 04는 부록 두 장을 참고하십시오. 05 품질 검사로 플래그 통계와 run_manifest를 남깁니다. 제출물은 9–10주차 학습 스크립트에 그대로 들어갑니다.

화면	설명
48:00 → 49:00	
 **#22 12주차 정리** 마무리 · 1분 49:00 → 50:00	정리하겠습니다. 첫째, 파이프라인은 6단계 루프이고 모듈끼리는 파일 · DB 레코드로만 주고받습니다. 둘째, 후보는 라이브러리 × 치환 레버 × 조성 격자로 만들고 제약 밖 조합은 생성 단계에서 뺍니다. 셋째, 조성 · 실행 · 물성 테이블과 실험 ID가 데이터베이스의 뼈대이며 계약을 통과한 값만 적재합니다. 넷째, 이상치는 지우지 않고 플래그로 처리하고, 결과는 코드 · 데이터 · 모델 · 실행을 함께 고정합니다. 13주차에는 이 격자와 데이터베이스를 입력으로 다목적 최적화, LLM 에이전트, 통합 사례를 다룹니다. 수고하셨습니다.
 **#23 부록** 부록 · 발표 시간 외	부록입니다. 계획서 차시에 맞지 않아 본문에서 뺀 두 장, 세 층위의 특징과 누수 방지입니다. 실습 03 · 04에 관한 질문이 나오면 이 두 장으로 설명합니다.

화면	설명
 **#24 부록 · 기술자** 부록 · 발표 시간 외	특징을 어떻게 고르느냐는 질문이 나오면, 표의 층위로 설명합니다. 조성(SARA 분율 · 분자별 mol% · 첨가제 wt%)은 해석이 쉽고 설계 변수와 바로 이어지고, 분자(평균 분자량 · 방향족 탄소 분율 · H/C · 극성 원자 수 · TPSA)는 새 분자로 확장할 수 있으며, 조건(온도 · 계 크기 · 힘장 · 티어)은 라벨끼리 비교할 수 있는지를 정합니다. RDF 첫 피크 · 자유부피 분율 같은 구조 특징은 궤적이 있어야 계산됩니다. 가장 중요한 것은 금지 줄입니다. 예측하려는 물성에서 유도한 값은 특징에 넣지 않습니다. 점도를 예측하면서 확산계수를 넣으면 평가는 좋아 보이지만 새 조성에서는 그 값도 계산해야 하므로 쓸 수 없는 모델이 됩니다. 10주차 GNN은 분자 수준 특징을 손으로 정하는 대신 학습으로 대체한 것이라고 연결해 줍니다.
 **#25 부록 · 누수 방지** 부록 · 발표 시간 외	누수(leakage)는 검증 데이터의 정보가 학습에 새어 들어가 성능이 부풀려지는 것입니다. 위 세 카드, 전체 데이터로 정규화하기, 같은 조성의 시드를 학습 · 검증에 나누기, 같은 조성의 다른 온도 행을 양쪽에 두기가 대표적입니다. 해결은 아래 흐름대로 comp_id로 묶어 조성 단위로 나누고, 학습 폴드에서만 μ · σ를 구해 검증 폴드에 같은 값을 적용하는 순서를 지키는 것입니다. **【콘크리트 예시】** 같은 배합 · 같은 배치로 만든 공시체들을 학습과 검증에 나눠 넣으면, 모델은 배합을 배운 것이 아니라 그 배치를 외운 셈입니다. 배합 ID로 묶어 나눠야 새 배합에 대한 성능을 제대로 잴 수 있습니다.